Learnings Document

Image Authentication Project

Tushar Yadav

Contents

1 Project Context and High-Level Goal **4**

2 Formal Problem Definition: Invariance vs Sensitivity **4**

2.1 Benign Invariance 4

2.2 Malicious Sensitivity 5

3 Images as Signals and Blocks **5**

3.1 Image as a Matrix 5

3.2 Block-Based Processing 5

4 DFT vs DCT and the Periodicity Confusion **5**

4.1 DFT and Periodicity 6

4.2 DCT vs DFT 6

4.3 Takeaway from the Confusion 6

5 2D DCT and Frequency Interpretation **7**

5.1 2D DCT Formula 7

5.2 Interpretation of (u, v) 7

6 Zig-Zag Ordering and "Low to High Frequency" **7**

6.1 Why Zig-Zag Exists 7

6.2 Zig-Zag Pattern 7

6.3 Key Understanding 8

7 JPEG Compression and Quantization **8**

7.1 Quantization Formula 8

7.2 Effects on the Image . . . 8
7.3 Effect on Authentication . . . 8

8 Lin & Chang's DCT-Based Image Authentication 9
8.1 High-Level Idea . . . 9
8.2 Block Pairing . . . 9
8.3 Feature Extraction Structure . . . 10
8.4 Threshold Logic Details . . . 10

9 Bit Ordering and Index Flattening 10
9.1 Bit Generation Order . . . 10
9.2 Recovering Pair Index From Flat Index . . . 11
9.3 Important Debugging Lesson . . . 11

10 Authentication Workflow in Detail 12
10.1 At Sign/Enroll Time . . . 12
10.2 At Verification Time . . . 12

11 Global vs Per-Pair Mismatch Logic 12
11.1 Global Mismatch Rate . . . 12
11.2 Per-Pair Mismatch Counting . . . 13
11.3 Per-Pair Tolerance . . . 13
11.4 Decision Rule Variation . . . 13

12 Tampering Confidence Definition 13

13 Dataset Design and Generation 14
13.1 Motivation: Controlled Experiment . . . 14
13.2 Folder Structure . . . 14
13.3 Index-Based Classification . . . 14
13.4 Compressed Variants . . . 15
13.5 Identical Variants . . . 15
13.6 Tampered Variants . . . 15

14 Verification Script and Metrics 16
14.1 Per-Image Evaluation . . . 16
14.2 Ground Truth Expectation . . . 16
14.3 Result Logging . . . 16

14.4 Confusion Matrix 17

15 Empirical Observations and Behavior **17**
15.1 Initial Results 17
15.2 Interpretation of Poor Tamper Detection 17
15.3 Parameter Tightening and Improvements 18

16 Identified Weaknesses and Possible Improvements **18**
16.1 Weaknesses of Original Approach 18
16.2 Adjacency-Based Pairing Idea 18
16.3 Deepfake-Oriented Extensions 19

17 Advanced Mathematical Insights and Security Analysis **19**
17.1 Why Multiple Thresholds Are Required 19
17.2 Block Size Tradeoff: Formal Analysis 20
17.3 Mathematical Structure of Threshold Binarization 20
17.4 Security Breakdown When Seed is Known 21
17.5 DCT Compensation Attack: Mathematical Formulation 21
17.6 Threshold Bin Attack 22
17.7 Why DCT is More Stable than DFT for Authentication 22
17.8 Connections to Related Modern Literature 22

1 Project Context and High-Level Goal

The project explores **image authentication** using feature mappings that are:

- **Invariant** to benign modifications (e.g., JPEG compression).
- **Sensitive** to malicious modifications (e.g., copy-paste tampering, deepfakes).

Formally, the idea is to design a function:

$$F : I \to Y$$

where:

- I is the space of images.
- Y is a feature space (e.g., bitstrings, vectors).
- $Y = F(I)$ is a compact descriptor capturing image content in an authentication-friendly way.

The descriptor must behave differently in two cases:

- **Benign variants** I_k: compressed, slightly denoised, brightness changes etc.
- **Malicious variants** I_m: semantically altered content (splicing, face replacement, deepfakes).

2 Formal Problem Definition: Invariance vs Sensitivity

We consider a distance metric d_Y in the feature space Y.

Let I_0 be the original image, I_k a benignly modified version, and I_m a maliciously modified version.

We want two inequalities:

2.1 Benign Invariance

For benign transformations,

$$d_Y(F(I_0), F(I_k)) < \epsilon$$

for a small tolerance ϵ. This means that benign operations (like JPEG compression) should *not* push the descriptor too far away.

2.2 Malicious Sensitivity

For semantic / malicious transformations,

$$d_Y(F(I_0), F(I_m)) \geq \delta$$

with

$$\delta \gg \epsilon.$$

This separation of scales (ϵ vs δ) is the theoretical backbone of the project: the feature mapping should compress benign noise into a small region while amplifying malicious changes into a large distance.

The Lin & Chang algorithm is an instance of such an F, using DCT coefficient relationships.

3 Images as Signals and Blocks

3.1 Image as a Matrix

A grayscale image is represented as:

$$I \in \mathbb{R}^{H \times W}$$

with pixel intensities typically in $[0, 255]$.

For processing, it is often convenient to see it as a 2D discrete signal $I[x, y]$.

3.2 Block-Based Processing

The image is divided into non-overlapping 8×8 blocks:

$$\text{Number of blocks} = \frac{H}{8} \cdot \frac{W}{8}$$

assuming H and W are multiples of 8.

Each block is processed independently using the 2D Discrete Cosine Transform (DCT). This is how JPEG works, and also how the Lin & Chang method works.

4 DFT vs DCT and the Periodicity Confusion

At one point, there was conceptual confusion about DFT vs DCT and the notion of periodicity.

4.1 DFT and Periodicity

The Discrete Fourier Transform (DFT) of a length-N sequence $x[n]$ is:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}.$$

- Mathematically, one can interpret $x[n]$ as one period of a periodic signal of period N.
- Even if the original signal isn't physically periodic, DFT implicitly assumes periodic extension for reconstruction.
- The periodicity is a property of the *mathematical model*, not the original physical signal.

4.2 DCT vs DFT

DCT is related to DFT but uses only cosine basis functions with certain symmetry assumptions. Key differences:

- DFT uses complex exponentials; DCT uses only cosines (real-valued).
- DCT typically assumes *even* symmetry at the boundaries, which reduces edge discontinuities.
- In image compression, DCT is preferred due to real coefficients and energy compaction properties.

4.3 Takeaway from the Confusion

The important learning:

- **DFT and DCT do not require the input signal to actually be periodic;** periodicity is a modeling assumption used for reconstruction.
- The transform of any finite sequence is well-defined; how we interpret it in terms of periodic extension is a separate conceptual layer.

5 2D DCT and Frequency Interpretation

5.1 2D DCT Formula

For an 8×8 block $B(x, y)$, the 2D DCT is given by:

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{7} \sum_{y=0}^{7} B(x, y) \cos\left(\frac{(2x+1)u\pi}{16}\right) \cos\left(\frac{(2y+1)v\pi}{16}\right),$$

where

$$\alpha(0) = \frac{1}{\sqrt{8}}, \quad \alpha(k) = \frac{1}{2}\sqrt{\frac{2}{8}} = \frac{1}{2}, \quad k > 0.$$

5.2 Interpretation of (u, v)

- $C(0, 0)$: DC coefficient (average intensity of the block).
- Small u, v: low-frequency variations (smooth shading).
- Large u, v: high-frequency variations (edges, fine details, noise).

Thus, the DCT decomposes the block into spatial frequency components.

6 Zig-Zag Ordering and "Low to High Frequency"

6.1 Why Zig-Zag Exists

The DCT block is 8×8. Coefficients are naturally 2D. For storage and processing (and JPEG encoding), we want a 1D sequence ordered from "more important" (low frequency) to "less important" (high frequency).

Zig-zag scanning traverses the matrix diagonally starting from $(0, 0)$ (DC), moving roughly from low to high frequency.

6.2 Zig-Zag Pattern

The pattern visits indices in order of increasing $u + v$. For example:

- Sum $s = u + v = 0$: only $(0, 0)$ (DC).
- $s = 1$: $(0, 1), (1, 0)$.

- $s = 2$: $(0, 2), (1, 1), (2, 0)$.

Within each diagonal, the scan direction alternates to cover the matrix in a zig-zag manner. This approximates an ordering of increasing frequency.

6.3 Key Understanding

- Each DCT coefficient corresponds to a 2D spatial frequency.
- Zig-zag is a heuristic ordering that tends to put low-frequency components earlier and high-frequency components later.
- It is not a perfect monotonic ordering of frequency magnitude, but it is well-aligned with JPEG's idea of progressive encoding and zero-run-length coding.

7 JPEG Compression and Quantization

7.1 Quantization Formula

Given DCT coefficients $C(u, v)$ and quantization table $Q(u, v)$, JPEG computes:

$$C'(u, v) = \text{round}\left(\frac{C(u, v)}{Q(u, v)}\right).$$

This heavily attenuates higher-frequency coefficients where Q is large.

7.2 Effects on the Image

- Low-frequency coefficients remain relatively accurate.
- High-frequency coefficients may become zero.
- The reconstructed image loses small details but is visually similar.

7.3 Effect on Authentication

Lin & Chang exploit the fact that:

- JPEG recompression introduces *small, distributed* changes.
- Malicious tampering (copy-paste, local region modifications) causes *structured, localized* changes that can break particular DCT relationships.

Hence, the authentication scheme must be:

- Robust to quantization noise.
- Sensitive to structural changes in certain block relationships.

8 Lin & Chang's DCT-Based Image Authentication

8.1 High-Level Idea

The Lin & Chang scheme does *not* compare entire images. Instead, it compares **relationships** between DCT coefficients of paired blocks.

1. Divide the image into 8×8 blocks.
2. Pair blocks according to a secret (seeded) random permutation.
3. For each block pair, compare corresponding DCT coefficients.
4. Extract a binary feature vector (feature code) from these comparisons.
5. Digitally sign (or hash) this feature vector to form an authentication signature.

8.2 Block Pairing

Given N blocks, define indices $0, 1, \ldots, N-1$.

1. Shuffle these indices randomly using a fixed seed.
2. Take the first $N/2$ as set A, the next $N/2$ as set B.
3. Pair A[i] with B[i] to form block pairs.

This randomness (with secret seed) adds security: an attacker not knowing the pairing cannot easily forge consistent block relationships.

8.3 Feature Extraction Structure

For each threshold level k, each pair p, and each DCT coefficient index c (in zig-zag order):

- Compute difference:
$$d_{k,p,c} = C_a(c) - C_b(c).$$
- Apply a threshold rule:
$$b_{k,p,c} = f(d_{k,p,c}, T_k).$$

The thresholds T_k typically form a decreasing sequence, e.g.,:

$$T_k = \frac{T}{2^{k-1}} \quad (k > 0), \quad T_0 = 0.$$

8.4 Threshold Logic Details

For $k = 0$ (sign-only):

$$b_{0,p,c} = \begin{cases} 1 & \text{if } d_{0,p,c} > 0, \\ 0 & \text{otherwise.} \end{cases}$$

For $k > 0$ (magnitude comparison):

$$b_{k,p,c} = \begin{cases} 1 & \text{if } |d_{k,p,c}| > T_k, \\ 0 & \text{otherwise.} \end{cases}$$

Thus, for each pair we get:

$$B_{\text{pair}} = \text{num\_thresholds} \times \text{num\_coefficients}$$

feature bits.

Collectively, all pairs form the global feature vector for the entire image.

9 Bit Ordering and Index Flattening

9.1 Bit Generation Order

The nested loops in the implementation generate bits in the order:

```
for k in range(num_thresholds):      # thresholds
```

```
for pair in range(num_pairs):    # block pairs
    for coeff in range(num_coefficients):  # DCT coeff index
        append bit
```

So the bit index i corresponds to a triple (k, p, c):

$$i = ((k \cdot P) + p) \cdot C + c$$

where:

- $P =$ number of pairs,
- $C =$ number of coefficients.

9.2 Recovering Pair Index From Flat Index

From:

$$i = ((k \cdot P) + p) \cdot C + c,$$

we have:

$$\left\lfloor \frac{i}{C} \right\rfloor = k \cdot P + p.$$

Therefore:

$$p = \left(\left\lfloor \frac{i}{C} \right\rfloor \right) \bmod P.$$

In code:

```
pair_idx = (i // num_coefficients) % num_pairs
```

This correct formula was **derived and debugged** after initially using an incorrect grouping by num_thresholds $\times$ num_coefficients, which assumed a different bit layout (pair-major ordering).

9.3 Important Debugging Lesson

- Bit generation order matters for index mapping.
- You cannot guess the index formula; you must derive it from the actual nesting of loops.
- A wrong flattening assumption leads to mismapping mismatches to wrong block pairs.

10 Authentication Workflow in Detail

10.1 At Sign/Enroll Time

Given an original image I_0:

1. Divide into blocks, compute DCT for each, zig-zag flatten.
2. Pair blocks using seeded random shuffle.
3. For each threshold k, pair p, coefficient c, compute $b_{k,p,c}$.
4. Concatenate all bits into a feature vector $Y = F(I_0)$.
5. Generate a digital signature or hash of Y (e.g., SHA-256).

10.2 At Verification Time

Given a test image $\hat{I}$:

1. Repeat the same process (same seed, same parameters) to obtain $\hat{Y} = F(\hat{I})$.
2. Compare $\hat{Y}$ with stored Y bit by bit.
3. Count mismatches per bit, and additionally, per block pair.
4. Decide if the image is authentic or tampered based on mismatch statistics.

11 Global vs Per-Pair Mismatch Logic

11.1 Global Mismatch Rate

Define:

$$M = \text{total mismatches}, \quad N = \text{total bits}.$$

$$\text{mismatch\_rate} = \frac{M}{N}.$$

A naive decision rule might be:

$$\text{authenticated} \iff \text{mismatch\_rate} < \text{threshold}.$$

However, this has weaknesses:

- JPEG compression causes small, widespread changes (low mismatch in many places).
- Local tampering affects few blocks but can cause many mismatches in those blocks.
- A global rate may hide local clusters of discrepancies.

11.2 Per-Pair Mismatch Counting

Instead, per-pair mismatches:

$$M_p = \text{number of mismatches for pair } p.$$

We compute:

$$M_p = \sum_{k,c} \mathbf{1}\{b_{k,p,c} \neq \hat{b}_{k,p,c}\}.$$

This allows detection of *localized* tampering: if some pair(s) have M_p above tolerance, then tampering is suspected even if the global mismatch rate is modest.

11.3 Per-Pair Tolerance

Each pair has $B = \text{num\_thresholds} \cdot \text{num\_coefficients}$ bits. A per-pair tolerance might allow up to:

$$\text{tolerance\_per\_pair} \approx 0.10 \cdot B,$$

to account for JPEG rounding noise.

Pairs exceeding this threshold are flagged as tampered.

11.4 Decision Rule Variation

- If the number of tampered pairs is small ($\leq$ some percentage of total pairs), we may still accept as authentic (JPEG noise).
- If many pairs are tampered, reject as malicious.

This logic evolved over time while debugging experiment behavior.

12 Tampering Confidence Definition

A heuristic:

$$\text{tampering\_confidence} = \min\left(\frac{\text{num\_tampered\_pairs}}{\text{num\_pairs}}, 1.0\right)$$

or earlier:

$$\text{tampering\_confidence} = \min\left(\frac{\text{mismatch\_rate}}{\text{auth\_threshold}}, 1.0\right).$$

Interpretation:

- Near 0: almost no evidence of tampering.
- Near 1: strong evidence of tampering.

This is not from the original paper; it is an added interpretability metric to quantify suspicion.

13 Dataset Design and Generation

13.1 Motivation: Controlled Experiment

To evaluate the algorithm, a controlled dataset was needed where we know:

- Which images are untouched (identical).
- Which are only JPEG compressed.
- Which are maliciously tampered with.

13.2 Folder Structure

At the code level:

- `input_images/` : original raw images (JPG).
- `generated/` : processed versions.

Images are renamed to:

$$\texttt{img1.jpg}, \texttt{img2.jpg}, \ldots$$

13.3 Index-Based Classification

Using integer index i from filename `img`i`.jpg`:

- If i is **prime** $\Rightarrow$ marked as "compressed".
- If i is **multiple of 5 but not 5 itself** $\Rightarrow$ marked as "identical".
- Otherwise $\Rightarrow$ marked as "tampered".

Prime and multiple-of-5 sets are precomputed up to 300 to avoid runtime primality testing.

13.4 Compressed Variants

For primes:

$$\texttt{img}i\texttt{\_processed.jpg}$$

is created by saving:

- Format: JPEG
- Quality: 80

This produces realistic lossy compression.

13.5 Identical Variants

For indices in multiples of 5:

$$\texttt{img}i\texttt{\_processed.jpg}$$

is just a direct copy of `img`i`.jpg`.

These serve as *true authentic* samples that must always be accepted.

13.6 Tampered Variants

For all other indices, tampering is performed:

1. Choose a random **donor image** different from the source.
2. Compute source image size $H \times W$.
3. Choose patch height and width:

$$h \in [0.10H, 0.15H], \quad w \in [0.10W, 0.15W].$$

4. Select a random $h \times w$ patch in the donor.
5. Paste that patch into a random location in the source.

This simulates realistic splicing / content insertion.

14 Verification Script and Metrics

14.1 Per-Image Evaluation

For each processed image:

1. Extract features from the original image.
2. Generate signature from the original.
3. Authenticate the processed image using this signature.
4. Retrieve:
    - Authentication result (True/False).
    - Mismatch rate.
    - Tampering confidence.
    - Number of manipulated pairs.
5. Compare actual authentication result with expected behavior based on ground truth (index-based).

14.2 Ground Truth Expectation

- For **identical** and **compressed**: we expect `authenticated` = `True`.
- For **tampered**: we expect `authenticated` = `False`.

14.3 Result Logging

For each image, information is stored:

- Index, ground truth label.
- Actual auth decision.
- Whether result is correct.
- Mismatch rate, tampering confidence.
- Number of manipulated pairs.

A JSON file (`verification_results.json`) is written with all results.

14.4 Confusion Matrix

From results, we compute:

- TP (True Positives): tampered images correctly rejected.
- FN (False Negatives): tampered images incorrectly accepted.
- FP (False Positives): authentic images incorrectly rejected.
- TN (True Negatives): authentic images correctly accepted.

Performance measures:

$$\text{Tampering Detection Rate (Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$
$$\text{False Rejection Rate (FPR)} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

These help diagnose behavior across categories (compressed, identical, tampered).

15 Empirical Observations and Behavior

15.1 Initial Results

Initially, empirical results showed:

- Near-100% correctness for identical images.
- Very high correctness for compressed images ($\sim 97\%$).
- Very poor performance ($\sim 30\%$ detection rate) for tampered images.

15.2 Interpretation of Poor Tamper Detection

Reasoning:

- Only a small fraction of blocks are inside the tampered region.
- Random block pairing may not involve many of these tampered blocks.
- Global mismatch thresholds may be too lenient, allowing small clusters of mismatches to be ignored.

In other words, the Lin & Chang method in its basic form is:

- Good at distinguishing JPEG vs original.
- Weak against small, localized tampering when pairing is random.

15.3 Parameter Tightening and Improvements

After making the authentication conditions stricter (e.g., reduced tolerance, stricter per-pair thresholds), performance improved:

- Tampering recall increased.
- False rejections for authentic images remained low.

This demonstrates that threshold-tuning is crucial.

16 Identified Weaknesses and Possible Improvements

16.1 Weaknesses of Original Approach

- Random block pairing may miss tampered blocks entirely if the tampered region is small.
- The method does not directly model spatial adjacency.
- Designed mainly for JPEG vs malicious editing, not for modern deepfake artifacts.

16.2 Adjacency-Based Pairing Idea

One proposed improvement:

- Pair each block with its spatial neighbors (right, bottom, etc.).
- Local tampering that affects a region will naturally disturb multiple adjacent relationships.
- This yields better sensitivity to localized tampering.

16.3 Deepfake-Oriented Extensions

Future work directions include:

- Moving from hand-crafted DCT features to learned feature mappings F (e.g., CNN embeddings).
- Maintaining invariance to benign transforms while amplifying inconsistencies in facial regions.
- Designing descriptors that react to semantic changes (identity, expression) rather than just low-level DCT differences.

17 Advanced Mathematical Insights and Security Analysis

17.1 Why Multiple Thresholds Are Required

The algorithm uses a sequence of thresholds:

$$T_0 = 0, \qquad T_k = \frac{T}{2^{k-1}},\ k \geq 1.$$

JPEG introduces quantization noise:

$$|\Delta C| \leq \frac{Q}{2},$$

which may be large enough to flip a single-threshold decision.

Multiple thresholds create **coarse-to-fine bins** so that:

$$\text{JPEG noise} < T_K \ll \text{tampering effect}.$$

Thus:

- Benign changes keep coefficient differences in the **same bin**.
- Malicious changes jump across **multiple bins** $\rightarrow$ many bit flips.

This is the mathematical basis for robustness.

17.2 Block Size Tradeoff: Formal Analysis

Let a tampered region have area A_t. Each block has area B^2. Then the approximate number of affected blocks is:

$$r \approx \frac{A_t}{B^2}.$$

Per-block disturbance energy is proportional to:

$$E_{\text{block}} \propto B^2.$$

Hence:

- **If blocks are too large:** small tampering spreads out $\rightarrow$ low sensitivity.
- **If blocks are too small:** DCT becomes unstable under JPEG noise $\rightarrow$ high false positives.

Conclusion:

$$8 \times 8 \text{ is near-optimal for balancing sensitivity and robustness.}$$

17.3 Mathematical Structure of Threshold Binarization

For each block pair p and coefficient c:

$$\Delta_c^{(p)} = x_{A_p}(c) - x_{B_p}(c).$$

Threshold rules define bins:

$$b_{k,p,c} = \begin{cases} 1 & \Delta_c^{(p)} > 0,\ k = 0, \\ 1 & |\Delta_c^{(p)}| > T_k,\ k \geq 1, \\ 0 & \text{otherwise.} \end{cases}$$

This partitions the real line into regions where JPEG noise is contained but tampering crosses boundaries.

Thus benign modifications satisfy:

$$|\Delta_c^{\text{jpeg}} - \Delta_c^{\text{orig}}| \leq T_K,$$

while tampering produces:

$$|\Delta_c^{\text{tampered}} - \Delta_c^{\text{orig}}| \gg T_1.$$

17.4 Security Breakdown When Seed is Known

Block pairing is defined by a permutation π_s generated from seed s:

$$A_p = \pi_s(p), \qquad B_p = \pi_s(p + P).$$

If an attacker knows s, then:

- They know exactly which blocks are paired.
- They can avoid modifying those pairs.
- They can modify only unpaired regions $\rightarrow$ tampering becomes invisible.
- They can enforce constraints on paired blocks by editing pixel values.

Thus, secrecy of the seed is essential for security.

17.5 DCT Compensation Attack: Mathematical Formulation

Original difference vector:

$$\Delta^{(p)} = \mathbf{x}_{A_p} - \mathbf{x}_{B_p}.$$

After tampering, the attacker wants:

$$\tilde{\Delta}^{(p)} \approx \Delta^{(p)}$$

even though the image content has changed.

Define loss:

$$\mathcal{L} = \sum_c \left((\tilde{x}_{A_p}(c) - \tilde{x}_{B_p}(c)) - \Delta_c^{(p)} \right)^2.$$

Since DCT is linear:

$$\tilde{\mathbf{x}} = M \cdot \text{DCT}(\tilde{I}),$$

the attacker can compute gradients and update pixels:

$$\tilde{I} \leftarrow \tilde{I} - \eta \frac{\partial \mathcal{L}}{\partial \tilde{I}}.$$

After optimization:

- Visual tampering is preserved,

- DCT relationships are also preserved,
- Authentication incorrectly accepts the tampered image.

This demonstrates the scheme is **not secure** if the seed is known.

17.6 Threshold Bin Attack

The attacker only needs to keep:

$$|\tilde{\Delta}_c| \in (T_k, T_{k-1}]$$

for the same threshold bin as the original image.

This is much easier than preserving exact values, because:

- Threshold bins are wide,
- JPEG noise stays within the lowest bin,
- Tampered values can be numerically adjusted to remain inside bins.

Thus threshold-bin matching is a feasible attack.

17.7 Why DCT is More Stable than DFT for Authentication

DFT assumes periodic extension of the signal, which causes sharp discontinuities at block boundaries in images.

DCT instead assumes an **even-symmetric extension**, yielding smoother boundaries and improved numerical stability.

Thus:

- DCT coefficients change smoothly under JPEG compression.
- DFT coefficients react strongly to boundary artifacts.

Therefore, DCT produces more robust authentication features.

17.8 Connections to Related Modern Literature

Ahmed et al. (2010): Introduced Randomized Pixel Modulation:

$$I'(x, y) = I(x, y) + R_K(x, y),$$

and switched to wavelets. This protects against block-pair forgery attacks.

Korus (2017): Classified Lin & Chang as:

Semi-fragile Active Protection.

Explained the lack of industry adoption due to absence of camera-side signing hardware.

Alotaibi et al. (2023): Performed JPEG artifact analysis under **homomorphic encryption**. Showed block-based invariants are still meaningful in encrypted domain.

Erfurth (2024): Used quantization tables with powers of two:

$$Q = 2^k,$$

so JPEG becomes bitshift operations, enabling mathematically stable signatures.

These works demonstrate the enduring relevance of block-transform-based authentication.